TTD Arxitekturasi va API chaqiriqlari body { font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif; color: #1f2937; line-height: 1.6; margin: 48px; font-size: 14px; } h1, h2, h3 { color: #111827; margin-bottom: 8px; } h1 { font-size: 28px; margin-top: 0; } h2 { font-size: 20px; margin-top: 28px; border-bottom: 1px solid #e5e7eb; padding-bottom: 6px; } h3 { font-size: 16px; margin-top: 20px; } p, li { margin: 6px 0; } code, pre { font-family: "SFMono-Regular", Menlo, Monaco, Consolas, monospace; } pre { background: #f3f4f6; border: 1px solid #e5e7eb; border-radius: 8px; padding: 12px; white-space: pre-wrap; word-break: break-word; } .note { background: #eff6ff; border-left: 4px solid #2563eb; padding: 12px 14px; border-radius: 6px; } .warn { background: #fff7ed; border-left: 4px solid #ea580c; padding: 12px 14px; border-radius: 6px; } .flow { background: #f9fafb; border: 1px dashed #9ca3af; border-radius: 8px; padding: 14px; font-weight: 600; } ul { padding-left: 20px; }

TTD arxitekturasi qanday ishlaydi: ayniqsa API chaqiriqlari

Ushbu hujjat loyiha ichidagi amaldagi qatlamli tuzilma asosida yozildi. Reponing o’zida TTD degan alohida modul yoki papka topilmadi, lekin kod oqimi amalda Presentation -> Domain -> Data ko’rinishida ishlayotgani sababli hujjat shu arxitektura asosida tushuntirildi.

Hozirgi kod bazada to’liq ishlayotgan API oqimi asosan getTasks() orqali tashkil qilingan. Ya’ni hozir asosiy misol sifatida GET /api/tasks ishlatiladi.

1. Qisqa mazmun

Bu arxitekturada har bir qatlamning vazifasi aniq bo’linadi:

- Presentation foydalanuvchi interfeysi va holatni boshqaradi.
- Domain biznes mantiq va kontraktlarni belgilaydi.
- Data API, server va tashqi manbalar bilan gaplashadi.

UI (TasksPage) -> Cubit (TasksCubit) -> UseCase (GetTaskUsecase) -> Repository (TaskRepository) -> DataSource (TaskRemoteDataSourceImpl) -> Dio -> API -> javob qaytadi -> UI state yangilanadi

2. Papkalar bo’yicha tuzilma

- lib/features/tasks/presentation/: sahifa, widget va Cubit.
- lib/features/tasks/domain/: repository kontrakti va usecase.
- lib/features/tasks/data/: datasource va repository implementatsiyasi.
- lib/core/: umumiy xatoliklar, Either tipi, usecase bazasi va routing.

3. API chaqirig’i amalda qanday yuradi

3.1. Oqim qayerdan boshlanadi

TasksPage ochilganda u yerning o’zida kerakli objectlar zanjiri quriladi: TaskRemoteDataSourceImpl -> TaskRepositoryImple -> GetTaskUsecase -> TasksCubit. So’ng darhol ..getTask() chaqiriladi.

TasksCubit(
  usecase: GetTaskUsecase(

```
    taskRepository: TaskRepositoryImple(
      dataSource: TaskRemoteDataSourceImpl(),
    ),
  ),
)..getTask()
```

Bu yondashuvning ma’nosi shuki, UI API bilan to’g’ridan-to’g’ri gaplashmaydi. UI faqat Cubit bilan ishlaydi, qolgan qatlamlar pastga qarab uzatiladi.

3.2. Cubit state boshqaradi

TasksCubit ichida getTask() ishlaganda avval state loading bo’ladi, keyin usecase chaqiriladi. Natija muvaffaqiyatli bo’lsa success, xato bo’lsa error holatiga o’tadi.

```
emit(state.copyWith(status: TaskStatus.loading));
final result = await usecase.call(NoParams());

if (result.isRight) {
  emit(state.copyWith(status: TaskStatus.success, data: result.right));
} else {
  emit(state.copyWith(
    status: TaskStatus.error,
    errorText: result.left.message,
  ));
}
```

Hozirgi kodda foydalanuvchi loading holatini ko’rishi uchun sun’iy ravishda Future.delayed(Duration(seconds: 3)) ham qo’shilgan.

3.3. UseCase biznes kirish nuqtasi bo’ladi

GetTaskUsecase domain qatlamida turadi va presentation qatlamini repository kontrakti bilan bog’laydi. U API qanday ishlashini bilmaydi, faqat taskRepository.getTasks() ni chaqiradi.

3.4. Repository abstraksiya beradi

TaskRepository domain kontrakt hisoblanadi. Bu qatlamning foydasi shundaki, Cubit yoki usecase API’ga bog’lanib qolmaydi. Ertaga API o’rniga local cache ishlatilsa ham, interfeys bir xil qoladi.

```
abstract class TaskRepository {
  Future<Either<Failure, List>> getTasks();
}
```

3.5. Repository implementation real manbaga ulanadi

TaskRepositoryImple data qatlamida joylashgan. U datasourcedan natijani olib, yuqori qatlamga qaytaradi. Hozircha bu qatlam juda yupqa, lekin aynan shu joyda kerak bo’lsa:

- mapping qilish,
- cache va remote orasida tanlash,
- bir nechta manbani birlashtirish,

- response ni domain modelga o’tkazish

bajarilishi mumkin.

3.6. DataSource API bilan bevosita gaplashadi

API ga haqiqiy so’rov aynan TaskRemoteDataSourceImpl ichida yuboriladi:

final response = await Dio().get("http://localhost:1337/api/tasks");

Bu yerda oqim quyidagicha:

- Dio HTTP so’rov yuboradi.
- Agar status kodi 200-299 oralig’ida bo’lsa, response.data[’data’] olinadi.
- Muvaffaqiyat bo’lsa Right(...) qaytariladi.
- Xato bo’lsa Left(Failure) qaytariladi.

4. Nega Either ishlatilgan

Loyihada natija Either<Failure, Data> orqali qaytariladi:

- Right = muvaffaqiyatli natija
- Left = xatolik

Bu usulning foydasi shuki, metod doimo bitta tipdagi natija qaytaradi va yuqori qatlamlar try/catch bilan to’lib ketmaydi.

```
if (result.isRight) {
  // success data
} else {
  // failure message
}
```

5. Xatoliklar qanday ushlanadi

TaskRemoteDataSourceImpl ichida uch xil asosiy holat bor:

- DioException: server yoki request muammosi.
- SocketException: internet yo’q yoki tarmoq uzilgan.
- catch: kutilmagan umumiy xatolik.

```
} on DioException catch (error) {
  return Left(DioFailure(message: error.message));
} on SocketException catch (_) {
  return Left(NetworkFailure(message: "Internet o’chiq"));
} catch (error) {
  return Left(Failure(message: ’Something went wrong’));
}
```

Keyin bu xatolik Cubit ga chiqadi va u errorText orqali UI ga uzatiladi.

6. Hozirgi loyihadagi API oqimining kuchli tomonlari

- UI qatlam API detallarini bilmaydi.
- Biznes qoidalar usecase orqali ajratilgan.
- Xatoliklar yagona formatda qaytadi.
- Test yozish osonlashadi, chunki repository kontrakti bor.

7. Muhim kuzatuvlar

Hozir dependency injection alohida container yoki service locator orqali emas, to’g’ridan-to’g’ri TasksPage ichida qurilgan. Kichik loyiha uchun bu ishlaydi, lekin loyiha kattalashsa get_it, injectable yoki alohida composition root kerak bo’lishi mumkin.

- Datasource hozir List qaytaryapti, ya’ni tiplar hali kuchli emas.
- task_model.dart amaliy oqimda hali ishlatilmayapti.
- Repository qatlamida mapping deyarli yo’q, natija xom holda yuqoriga chiqyapti.
- API URL hardcoded: http://localhost:1337/api/tasks.

8. Yangi API qo’shmoqchi bo’lsak tartib qanday bo’ladi

Masalan, yangi POST /api/tasks qo’shish uchun odatda quyidagi tartib bo’ladi:

- Datasource ichida addTask(...) yoziladi va Dio orqali POST yuboriladi.
- Repository kontraktiga addTask metodi qo’shiladi.
- Repository implementation datasourcedagi yangi metodni chaqiradi.
- Domain qatlamida AddTaskUsecase yoziladi.
- Cubit ichida yangi action ochiladi.
- UI shu actionni chaqiradi.

UI -> Cubit.addTask() -> AddTaskUsecase -> TaskRepository.addTask() -> TaskRemoteDataSource.addTask() -> Dio().post(...) -> API

9. Qisqa misol: GET so’rov boshidan oxirigacha

- Foydalanuvchi TasksPage ga kiradi.
- BlocProvider ichida TasksCubit yaratiladi.
- ..getTask() ishga tushadi.
- Cubit state ni loading ga o’tkazadi.
- Cubit GetTaskUsecase ni chaqiradi.
- Usecase TaskRepository ga murojaat qiladi.
- Repository TaskRemoteDataSourceImpl ni chaqiradi.
- Datasource Dio orqali GET /api/tasks yuboradi.
- Serverdan javob keladi.
- Natija Right(data) yoki Left(failure) ko’rinishida qaytadi.
- Cubit yangi state beradi.
- UI state ga qarab ro’yxat yoki xatolikni ko’rsatadi.

10. Koddagi tayanch nuqtalar

- TasksPage: dependency zanjiri va birinchi chaqiriq.
- TasksCubit: state boshqaruvi.
- GetTaskUsecase: biznes kirish nuqtasi.
- TaskRepository: kontrakt.
- TaskRepositoryImple: repository implementatsiyasi.

- TaskRemoteDataSourceImpl: real HTTP so’rov joyi.
- Either va Failure: success/error formatlari.

11. Xulosa

Bu loyihadagi TTD arxitektura yondashuvi API chaqiriqlarini qatlamlarga ajratib boshqaradi. Natijada UI soddalashadi, biznes mantiq ajraladi va data manbasini almashtirish osonlashadi. Hozirgi implementatsiya sodda va tushunarli, ayniqsa birinchi clean architecture tajribasi uchun yaxshi asos bo’la oladi. Keyingi bosqichda typed model, dependency injection va request konfiguratsiyasini markazlashtirish qo’shilsa arxitektura ancha mustahkam bo’ladi.

12. Manba fayllar

- lib/features/tasks/presentation/pages/tasks_page.dart (17-21: dependency zanjiri va getTask start)
- lib/features/tasks/presentation/cubit/tasks_cubit.dart (15-24: state va usecase oqimi)
- lib/features/tasks/domain/usecases/get_task_usecase.dart (8-15: usecase delegatsiyasi)
- lib/features/tasks/domain/repositories/task_repository.dart (4-5: repository kontrakti)
- lib/features/tasks/data/repositories/task_repository_impl.dart (12-18: repository implementatsiyasi)
- lib/features/tasks/data/datasources/task_remote_data_source.dart (14-33: Dio so’rovi va error handling)
- lib/core/error/failures.dart (3-21: failure turlari)
- lib/core/utils/either.dart (9-60: Left/Right natija modeli)